

Self-Attention

Date: March 31, 2025

Mark Crowley

Sources

- **paper:** A. Vaswani *et al.*, "Attention is All you Need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.
- **preprint:** B. Ghogho and A. Ghodsi, "Attention mechanism, transformers, BERT, and GPT: Tutorial and survey." Dec. 2020. doi: [10.31219/osf.io/m6gcn](https://doi.org/10.31219/osf.io/m6gcn).

Variable Sizes and Meanings in the Original Transformer

- **Embedding Size** = $512 = d_{model}$
- **Context Size** = $64 = d_k = d_v = d_{model}/h$
- **Num Separate Attention Heads** = $8 = h$
 - **question:** why 8?
 - **response:** trial and error, but it does relate to parallelization. And the context size d_k is calculated in relation to this
- **Encoder-Decoder Stack Depth** = $6 = N$ (**why?**)
 - **response:** why 3x3 filters in CNNs? Why 3 max-pooling steps in CNNs?
 - then again, it is completely arbitrary, later models use many different stack depths.

Multi-Head Attention

"**Multi-Head Attention:** Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values."

In other words:

- Each word is converted to a common embedding space of size $d_{model} = 512$ in the original paper
- Instead of ...
 - learning correlations between these large vectors
 - they **project down** to a space which is just about **relations between** the words given as input, indexed using the current context size $d_k = 64$

The Attention Function

"**Attention:** An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where **the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.**"

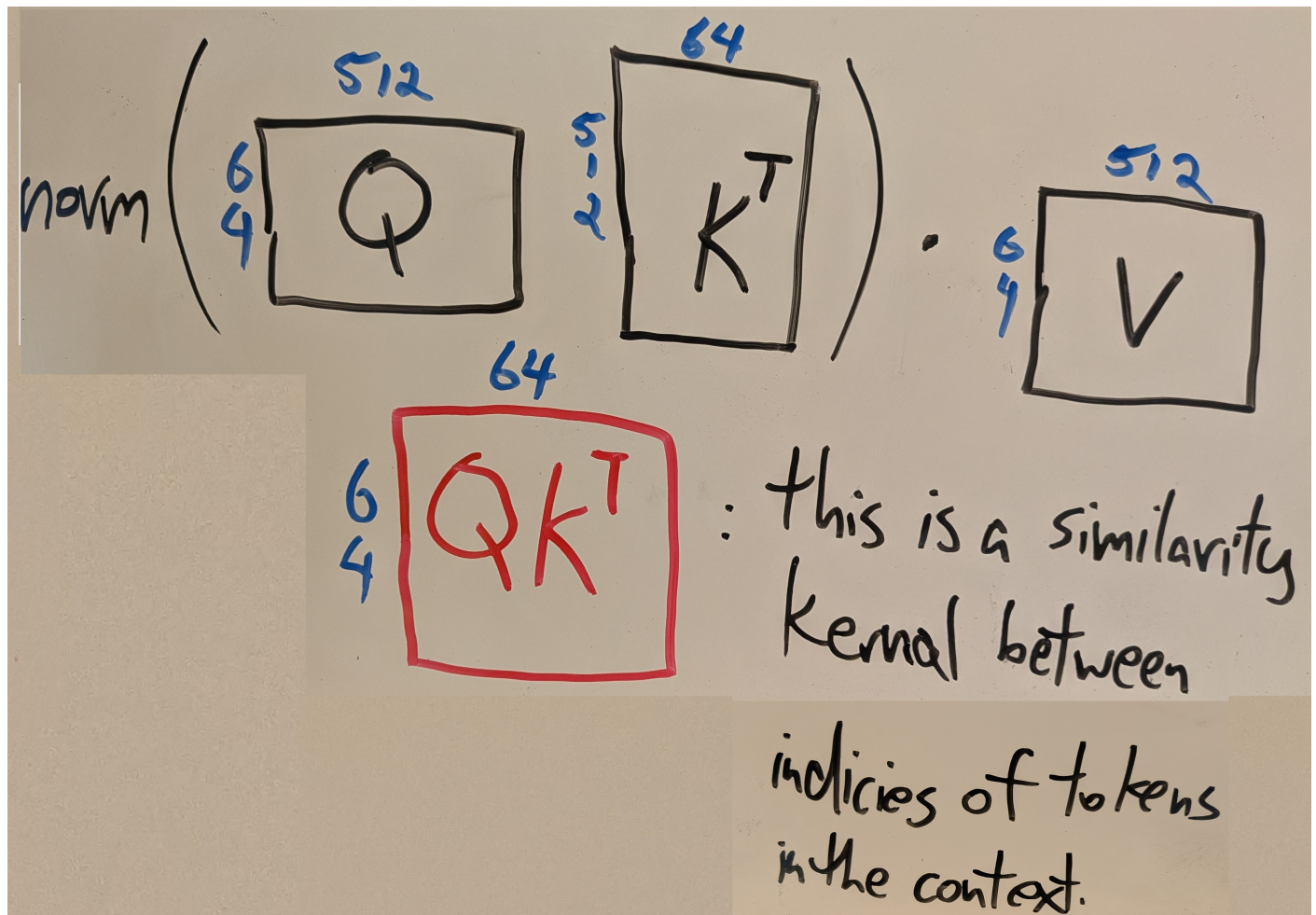
$$\text{attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

The Attention Function : Q, K

$$\text{attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Q, K : They aren't different really, they represent the same tokens in the context. By transposing K , we get to look at 64×64 correlations between those words in the context without using the full 512 embedding space.

But What do the matrices "Mean": Q,K?



The Attention Function : V

$$\text{attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- V : This matrix maps from the words in context (64 in this case) to the global representation of those words in the embedding space (512 in this case) or vocabulary.
- $\frac{1}{\sqrt{d_k}}$: they pre-emptively scale down the score since there will be 8 versions added together later, since there are 8 (in general h) attention heads each with their own independent set of weights for Q,K,V.

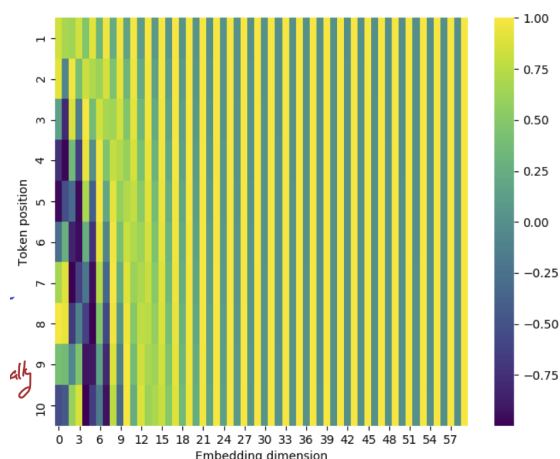
But What do the matrices "Mean": V,Z?

$$\begin{matrix} \text{output} \\ \times 32768 \end{matrix} \left(\begin{matrix} 64 \\ 64 \end{matrix} \begin{matrix} QK^T \end{matrix} \cdot \begin{matrix} 64 \\ 512 \end{matrix} \begin{matrix} V \end{matrix} \right) = Z$$

Z is the new score for how the $d_k (64)$ input tokens relate back to the whole $\mathbb{R}^{d_{\text{model}}} (\mathbb{R}^{512})$ representation space.

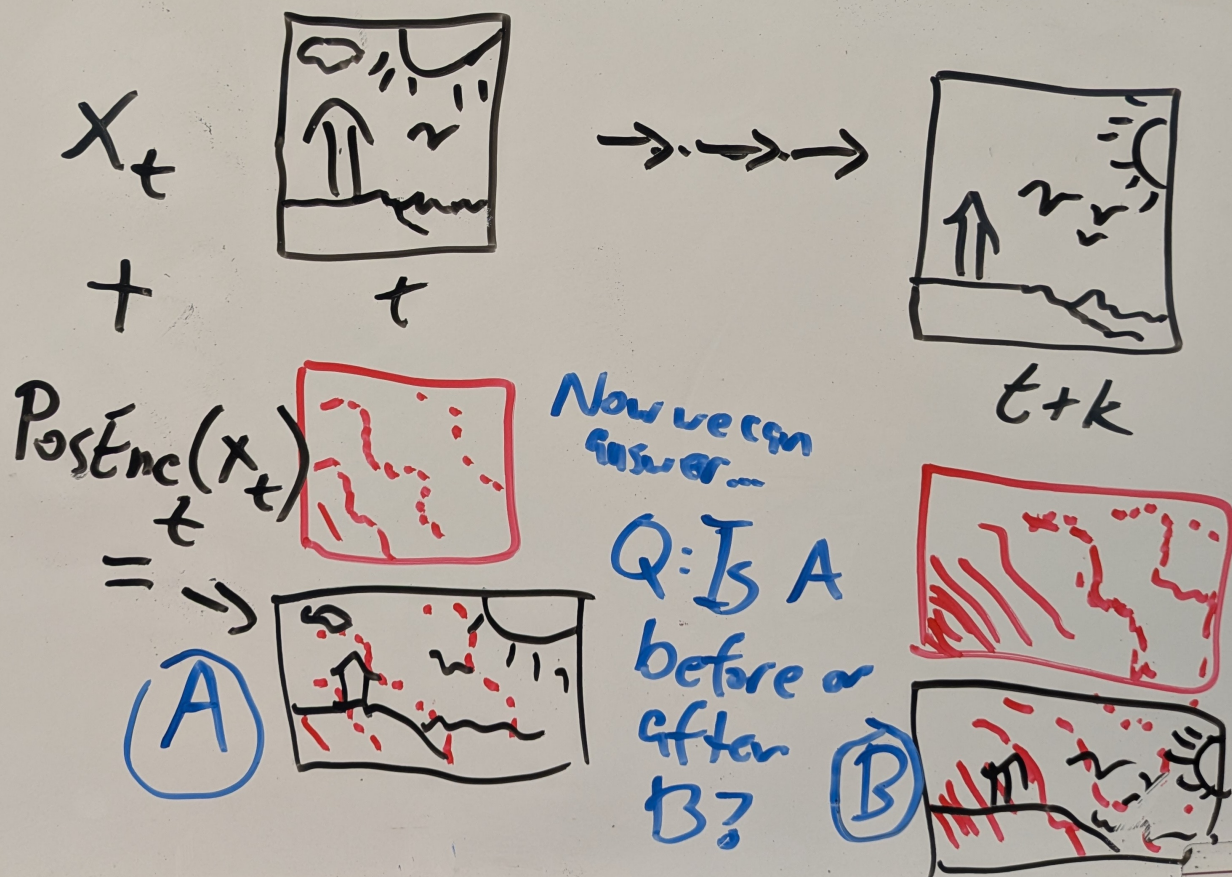
Positional Encoding

In the "[Attention Mechanism, Transformers, BERT, and GPT: Tutorial and Survey](#)" ([Ghojogh, 2023](#)) there is a detailed description of the original position encoding used for Transformers. It is a periodic pattern of numbers added on top of the encoded token input data that looks like this. It's hard to visualize, but this shows all the patterns that get added to each token (by row) for all their dimensions. The idea is that this code on top of the original representation, marks up the tokens by their relative position, which can be "seen" by the model when learning correlations between tokens.



A more intuitive way we could understand it is to think about this would be as readable watermarks on a series of images. We could train ourselves to read these watermark patterns and be able to tell what order two images were taken in, and how far apart they are.

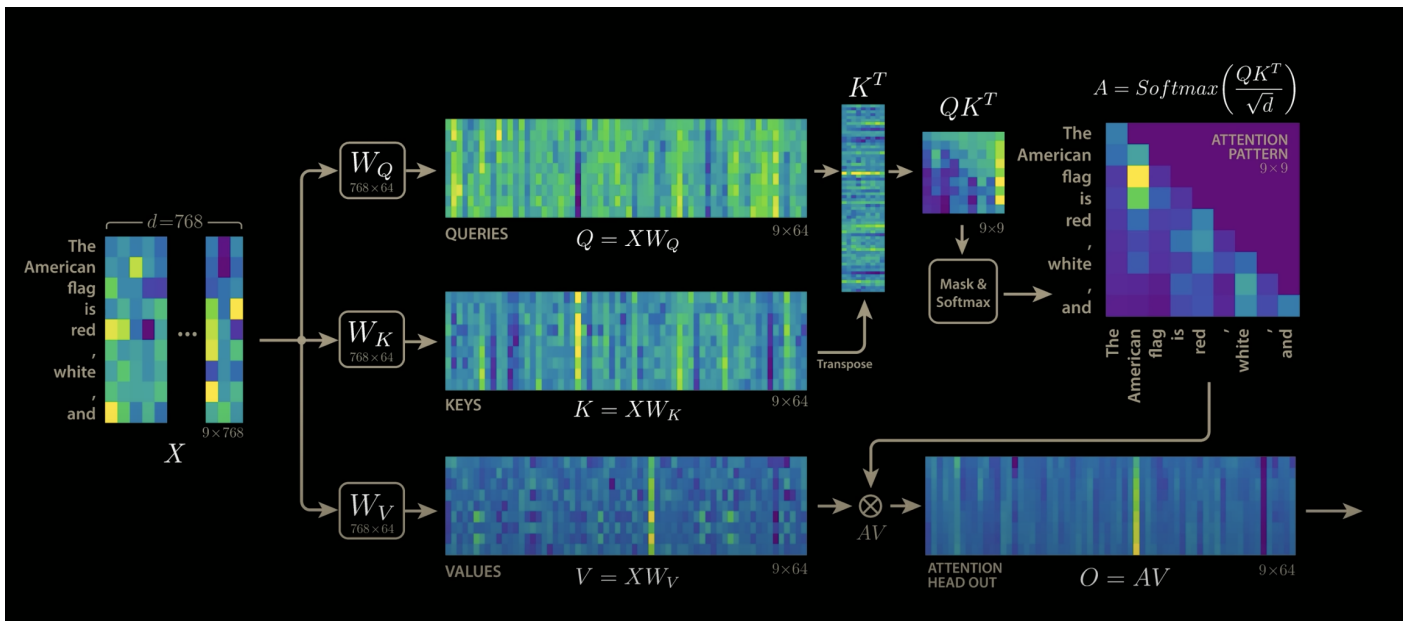
What is a Positional Encoding?



Even more directly, we could just print a transparent text timestamp in a corner of the image.

The Big Picture

Here's a nice overview visualization of the entire Self-Attention calculation. (note that the numbers don't match the Viswani 2017 paper)



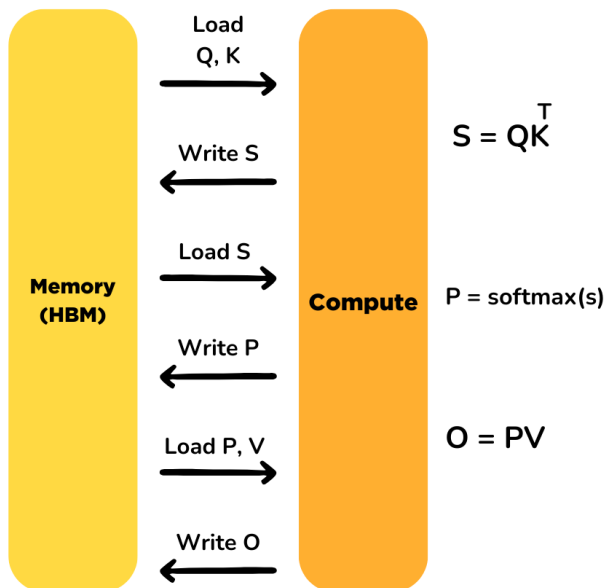
From Youtube: ["How DeepSeek Rewrote the Transformer"](#)

Flash Attention

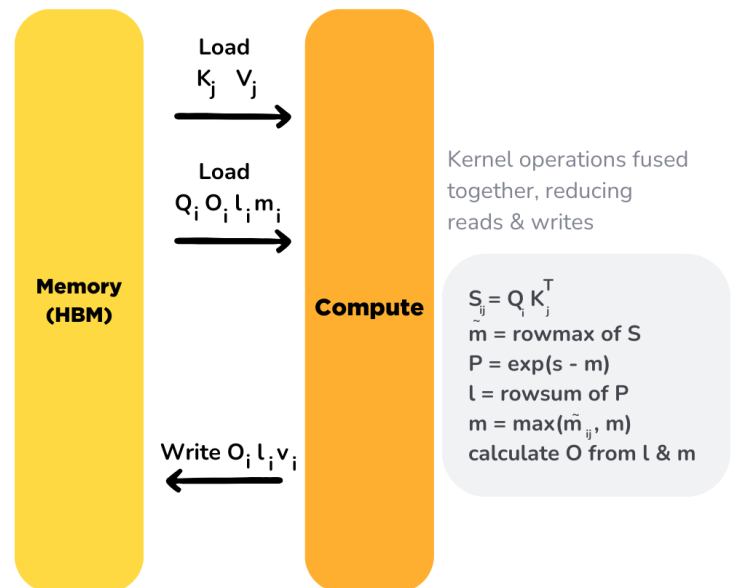
This is a computational efficiency improvement to sequential self-attention that isolates only the calculations that will actually change each step and loads those into fast memory for calculation. The original formulation of it unnecessarily loads the whole Q, K, V matrices into memory every time.

This figure from the ["Flash Attention" \(HuggingFace.co\)](#) page shows it clearly.

Standard Attention Implementation



Flash Attention



Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.